Progetti di Ingegneria del Software Tecniche Avanzate (ISTA)

A.A. 2026/27

Regole di Ingaggio

In questa presentazione vedremo alcune possibili idee di progetto, che consistono in manutenzione ed evoluzione di applicativi esistenti. Questi applicativi possono essere tool/script di analisi/manutenzione del codice e modelli predittivi per assistere attività di sviluppo.

Regole di Ingaggio

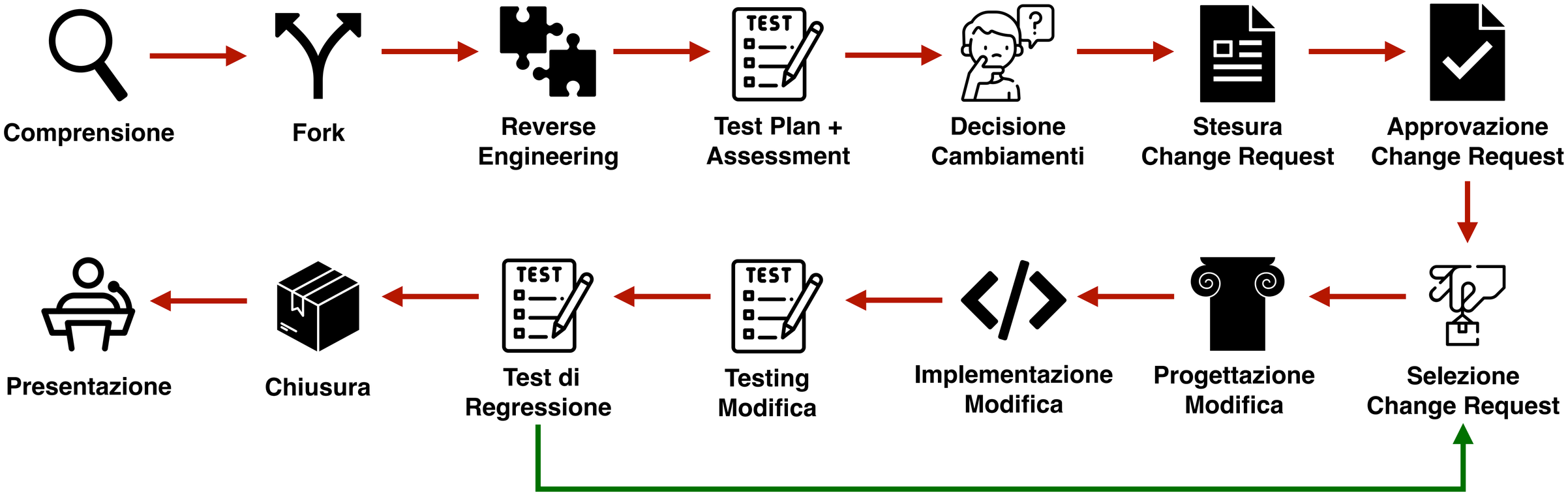
In questa presentazione vedremo alcune possibili idee di progetto, che consistono in manutenzione ed evoluzione di applicativi esistenti. Questi applicativi possono essere tool/script di analisi/manutenzione del codice e modelli predittivi per assistere attività di sviluppo.

Questo è il processo che si **dovrebbe** seguire per il completamento del progetto:

Regole di Ingaggio

In questa presentazione vedremo alcune possibili idee di progetto, che consistono in manutenzione ed evoluzione di applicativi esistenti. Questi applicativi possono essere tool/script di analisi/manutenzione del codice e modelli predittivi per assistere attività di sviluppo.

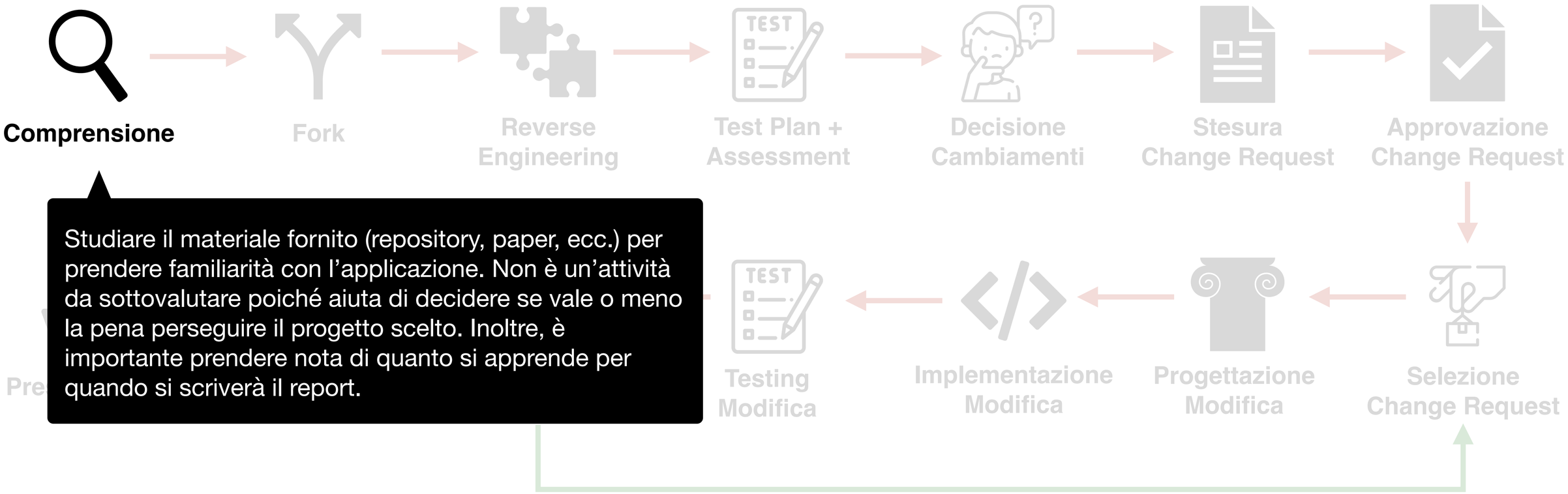
Questo è il processo che si **dovrebbe** seguire per il completamento del progetto:



Regole di Ingaggio

In questa presentazione vedremo alcune possibili idee di progetto, che consistono in manutenzione ed evoluzione di applicativi esistenti. Questi applicativi possono essere tool/script di analisi/manutenzione del codice e modelli predittivi per assistere attività di sviluppo.

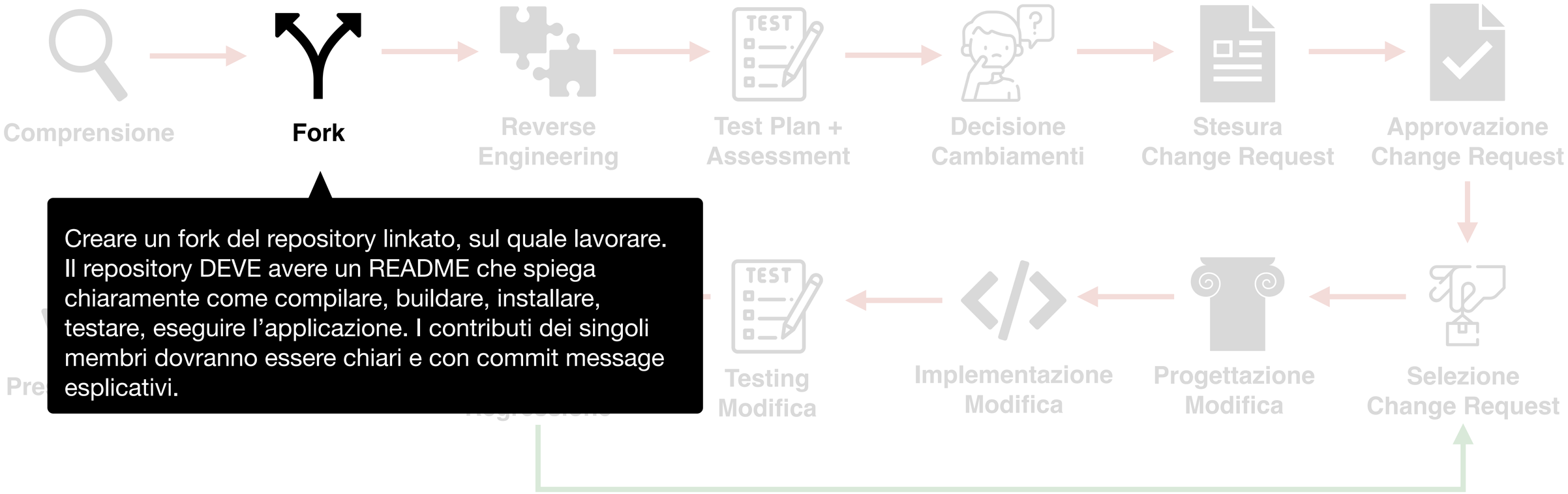
Questo è il processo che si **dovrebbe** seguire per il completamento del progetto:



Regole di Ingaggio

In questa presentazione vedremo alcune possibili idee di progetto, che consistono in manutenzione ed evoluzione di applicativi esistenti. Questi applicativi possono essere tool/script di analisi/manutenzione del codice e modelli predittivi per assistere attività di sviluppo.

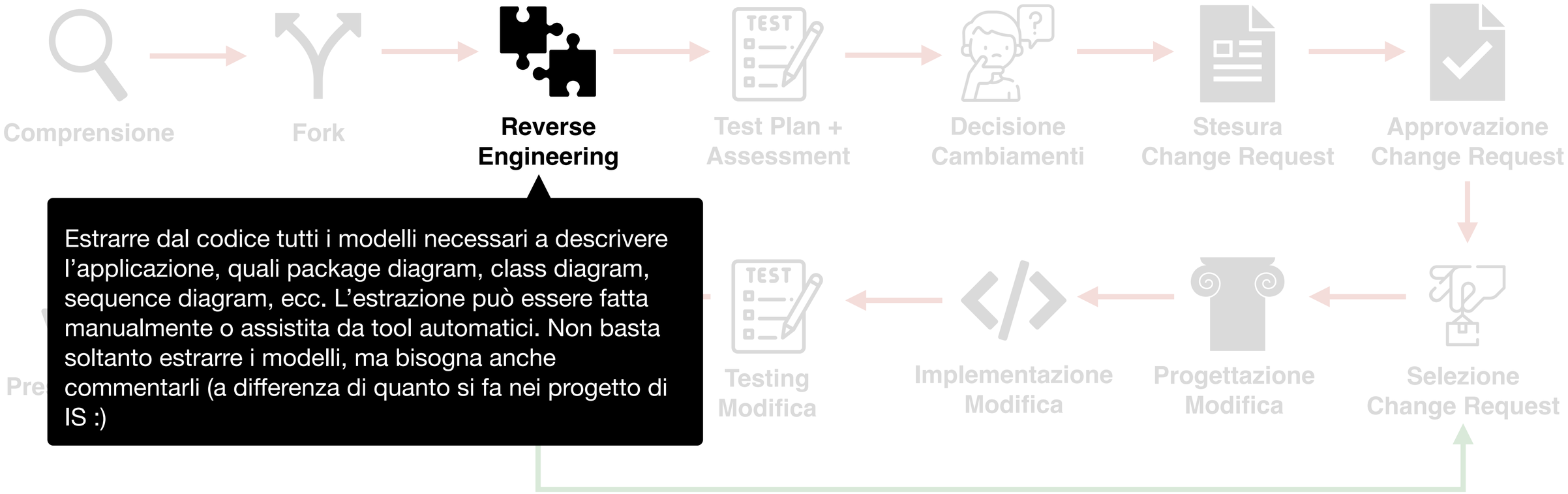
Questo è il processo che si **dovrebbe** seguire per il completamento del progetto:



Regole di Ingaggio

In questa presentazione vedremo alcune possibili idee di progetto, che consistono in manutenzione ed evoluzione di applicativi esistenti. Questi applicativi possono essere tool/script di analisi/manutenzione del codice e modelli predittivi per assistere attività di sviluppo.

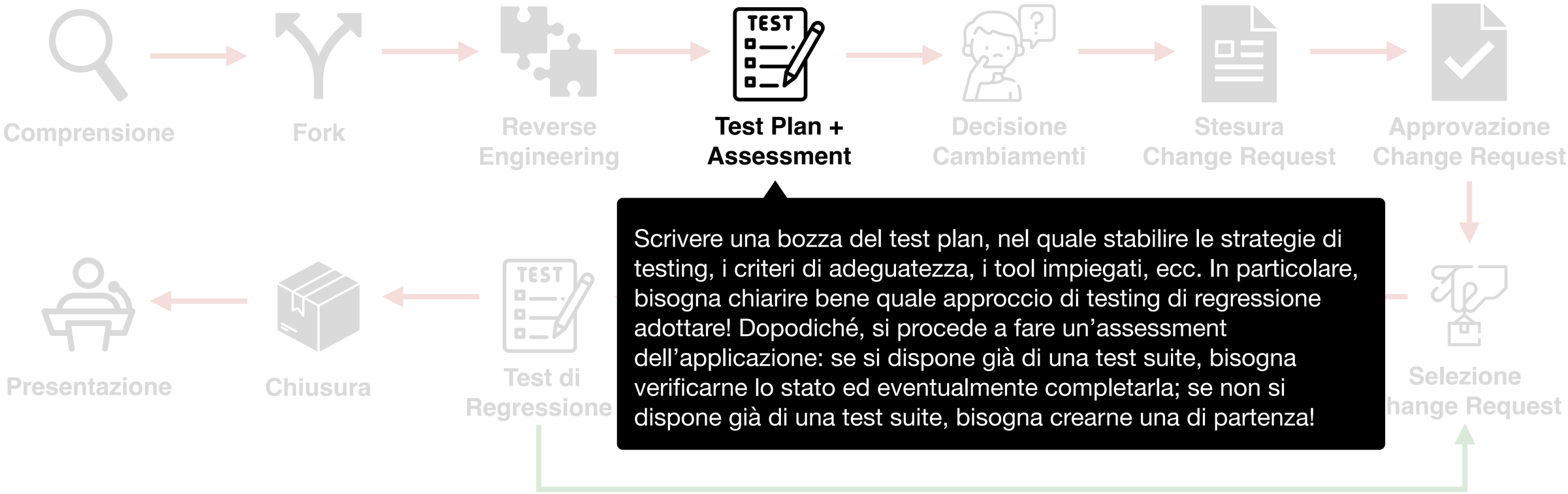
Questo è il processo che si **dovrebbe** seguire per il completamento del progetto:



Regole di Ingaggio

In questa presentazione vedremo alcune possibili idee di progetto, che consistono in manutenzione ed evoluzione di applicativi esistenti. Questi applicativi possono essere tool/script di analisi/manutenzione del codice e modelli predittivi per assistere attività di sviluppo.

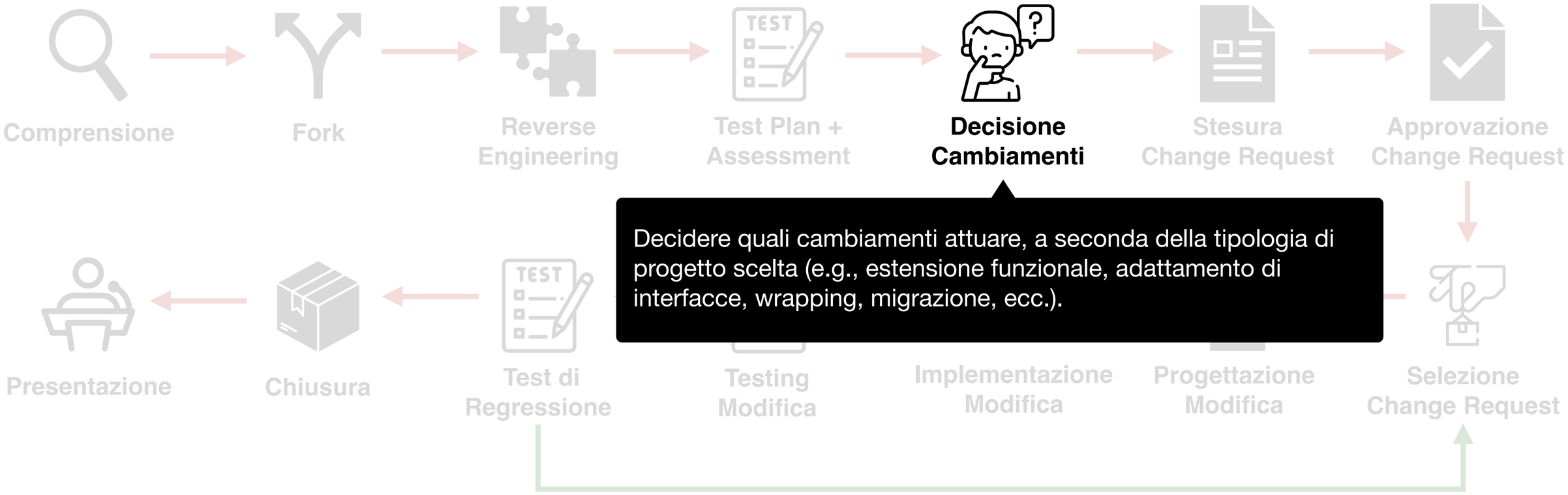
Questo è il processo che si **dovrebbe** seguire per il completamento del progetto:



Regole di Ingaggio

In questa presentazione vedremo alcune possibili idee di progetto, che consistono in manutenzione ed evoluzione di applicativi esistenti. Questi applicativi possono essere tool/script di analisi/manutenzione del codice e modelli predittivi per assistere attività di sviluppo.

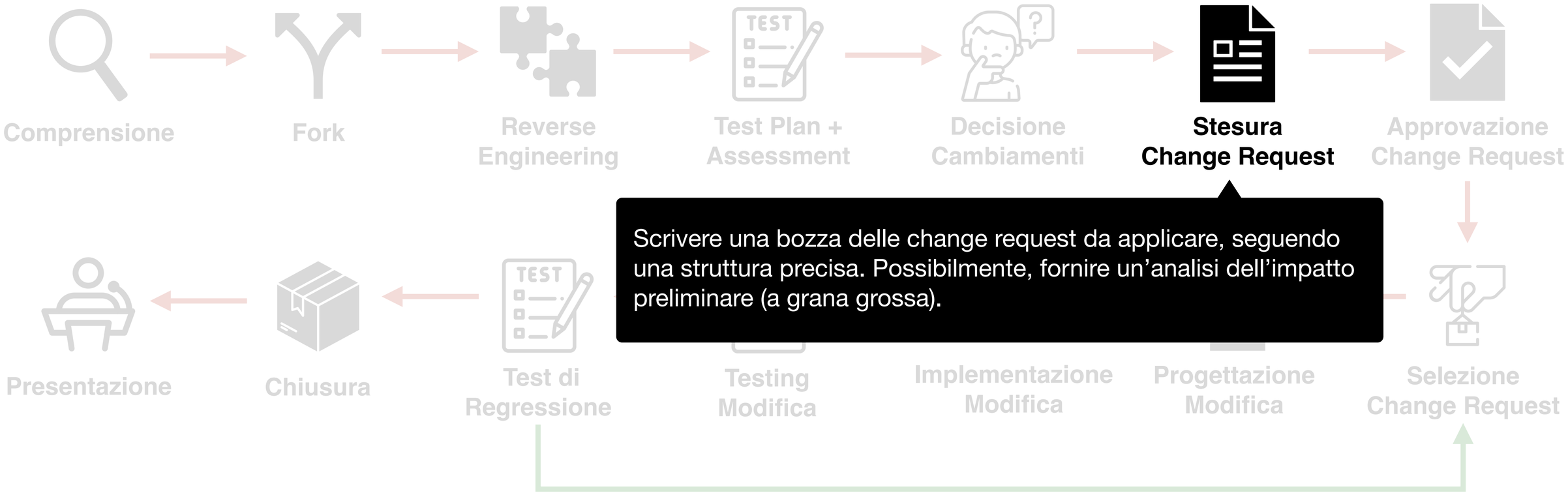
Questo è il processo che si **dovrebbe** seguire per il completamento del progetto:



Regole di Ingaggio

In questa presentazione vedremo alcune possibili idee di progetto, che consistono in manutenzione ed evoluzione di applicativi esistenti. Questi applicativi possono essere tool/script di analisi/manutenzione del codice e modelli predittivi per assistere attività di sviluppo.

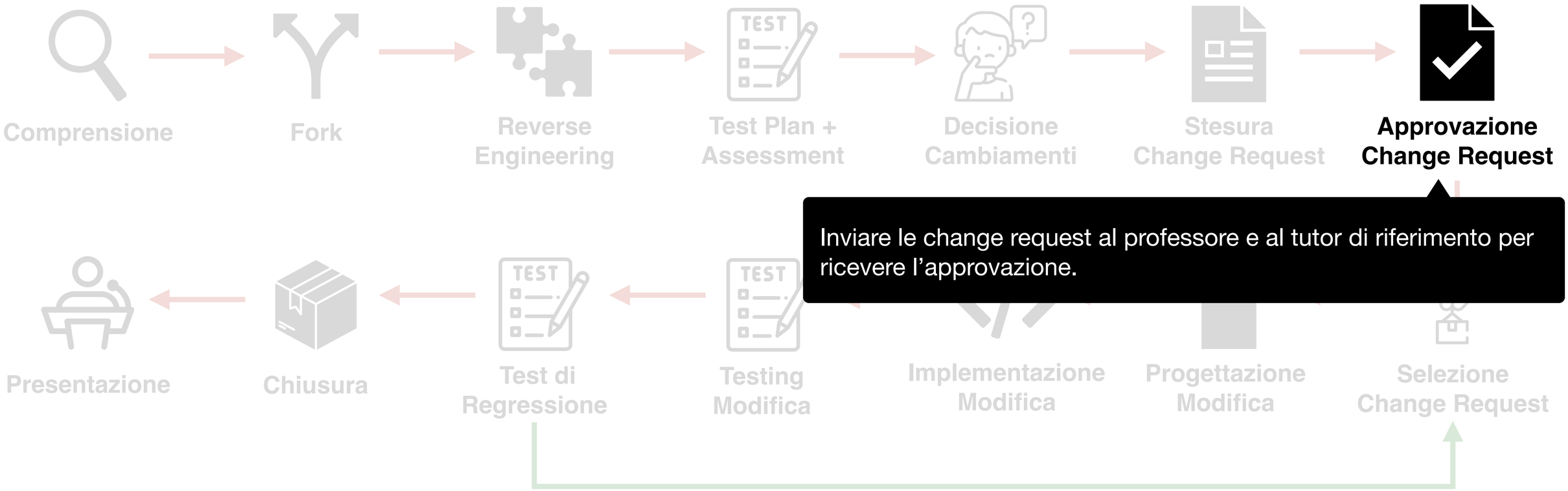
Questo è il processo che si **dovrebbe** seguire per il completamento del progetto:



Regole di Ingaggio

In questa presentazione vedremo alcune possibili idee di progetto, che consistono in manutenzione ed evoluzione di applicativi esistenti. Questi applicativi possono essere tool/script di analisi/manutenzione del codice e modelli predittivi per assistere attività di sviluppo.

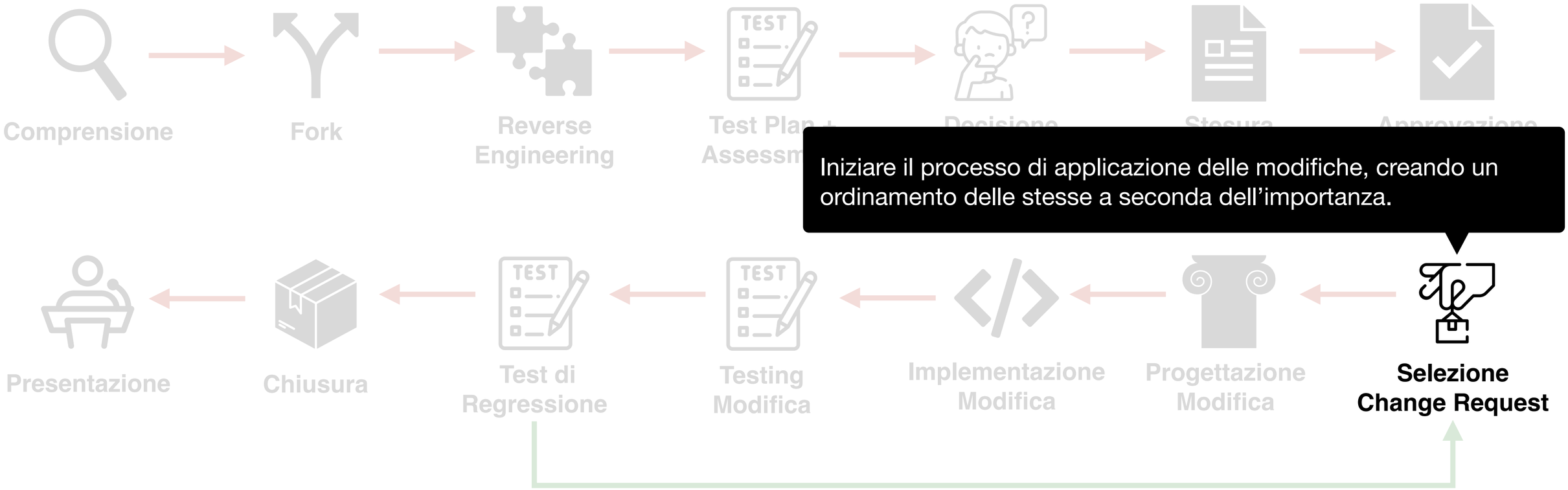
Questo è il processo che si **dovrebbe** seguire per il completamento del progetto:



Regole di Ingaggio

In questa presentazione vedremo alcune possibili idee di progetto, che consistono in manutenzione ed evoluzione di applicativi esistenti. Questi applicativi possono essere tool/script di analisi/manutenzione del codice e modelli predittivi per assistere attività di sviluppo.

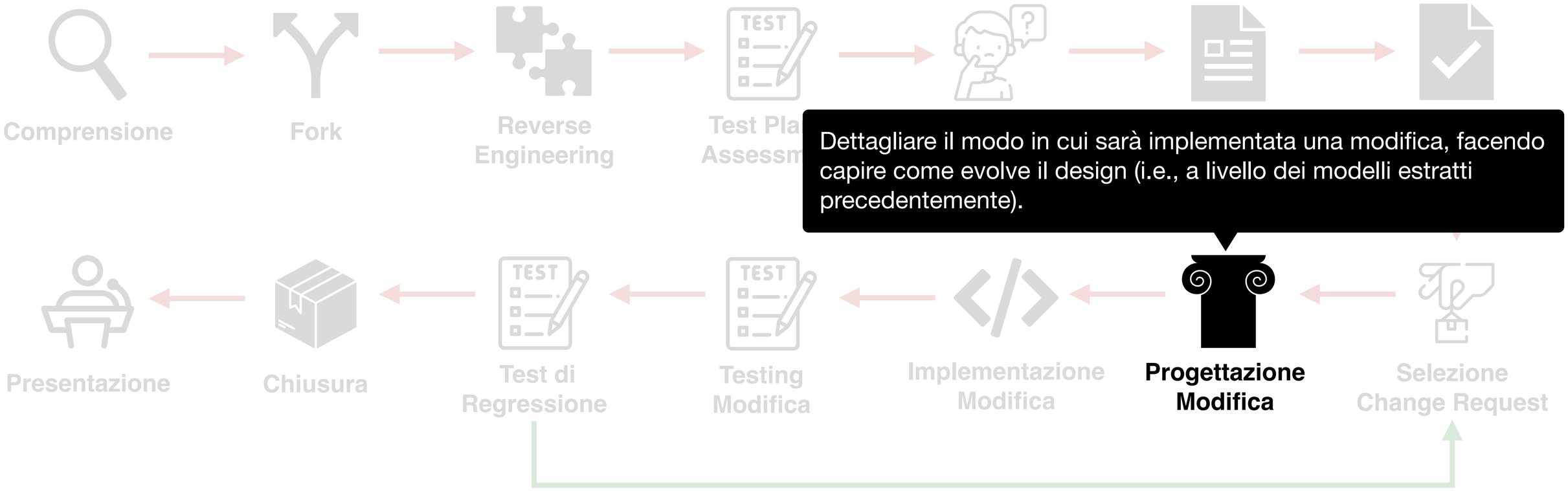
Questo è il processo che si **dovrebbe** seguire per il completamento del progetto:



Regole di Ingaggio

In questa presentazione vedremo alcune possibili idee di progetto, che consistono in manutenzione ed evoluzione di applicativi esistenti. Questi applicativi possono essere tool/script di analisi/manutenzione del codice e modelli predittivi per assistere attività di sviluppo.

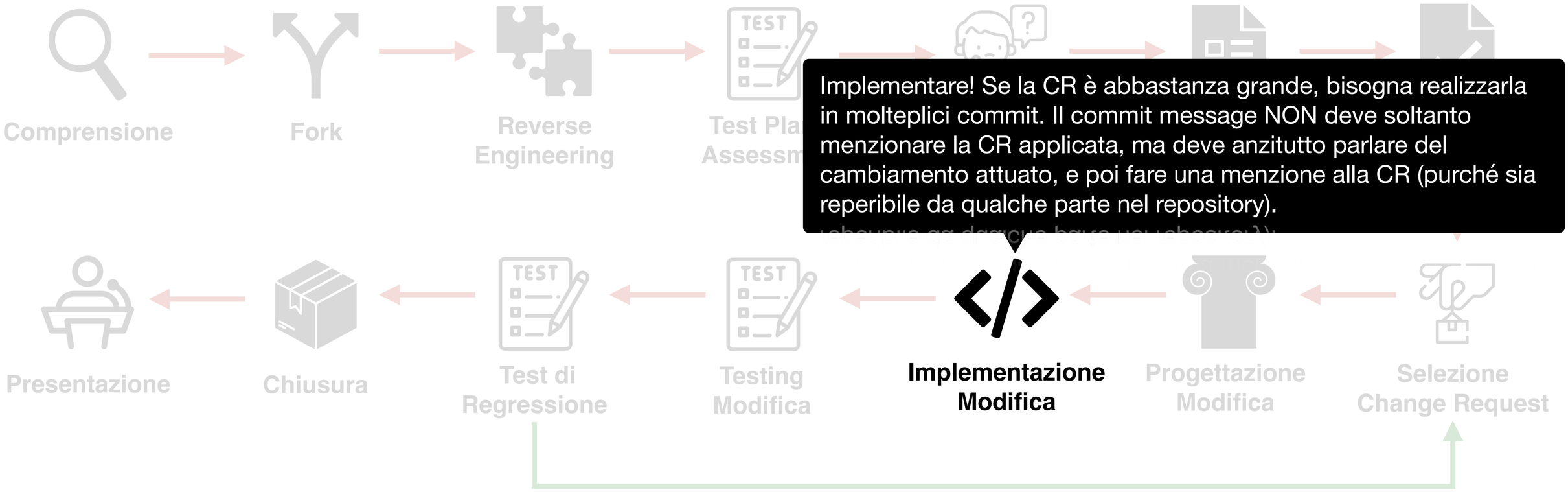
Questo è il processo che si **dovrebbe** seguire per il completamento del progetto:



Regole di Ingaggio

In questa presentazione vedremo alcune possibili idee di progetto, che consistono in manutenzione ed evoluzione di applicativi esistenti. Questi applicativi possono essere tool/script di analisi/manutenzione del codice e modelli predittivi per assistere attività di sviluppo.

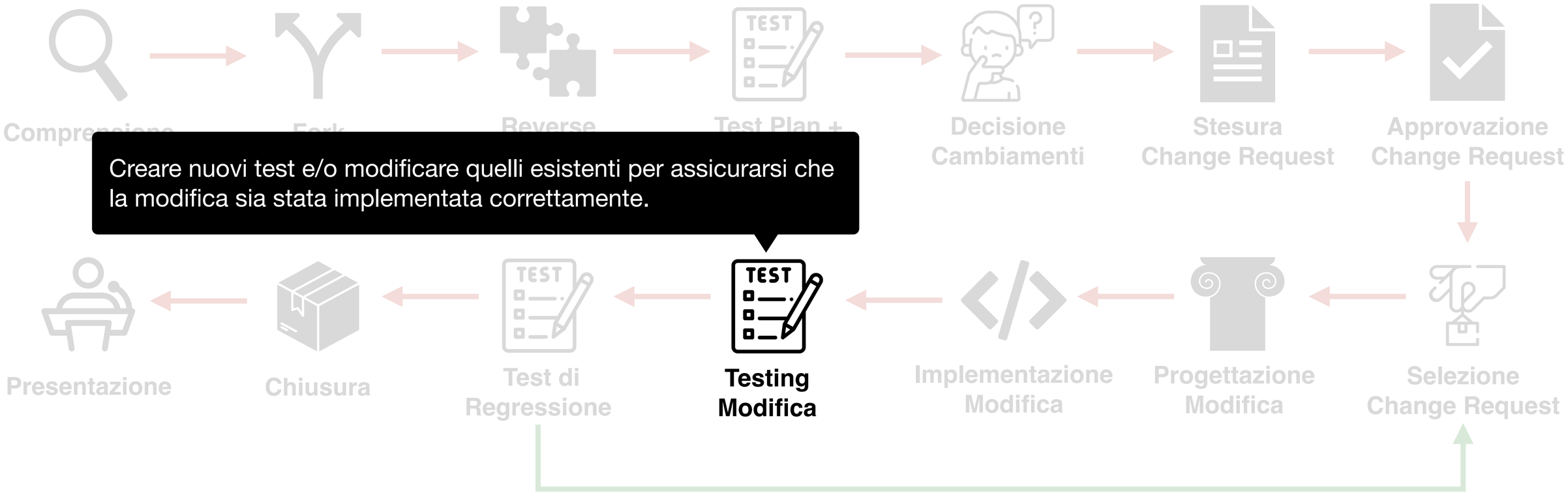
Questo è il processo che si **dovrebbe** seguire per il completamento del progetto:



Regole di Ingaggio

In questa presentazione vedremo alcune possibili idee di progetto, che consistono in manutenzione ed evoluzione di applicativi esistenti. Questi applicativi possono essere tool/script di analisi/manutenzione del codice e modelli predittivi per assistere attività di sviluppo.

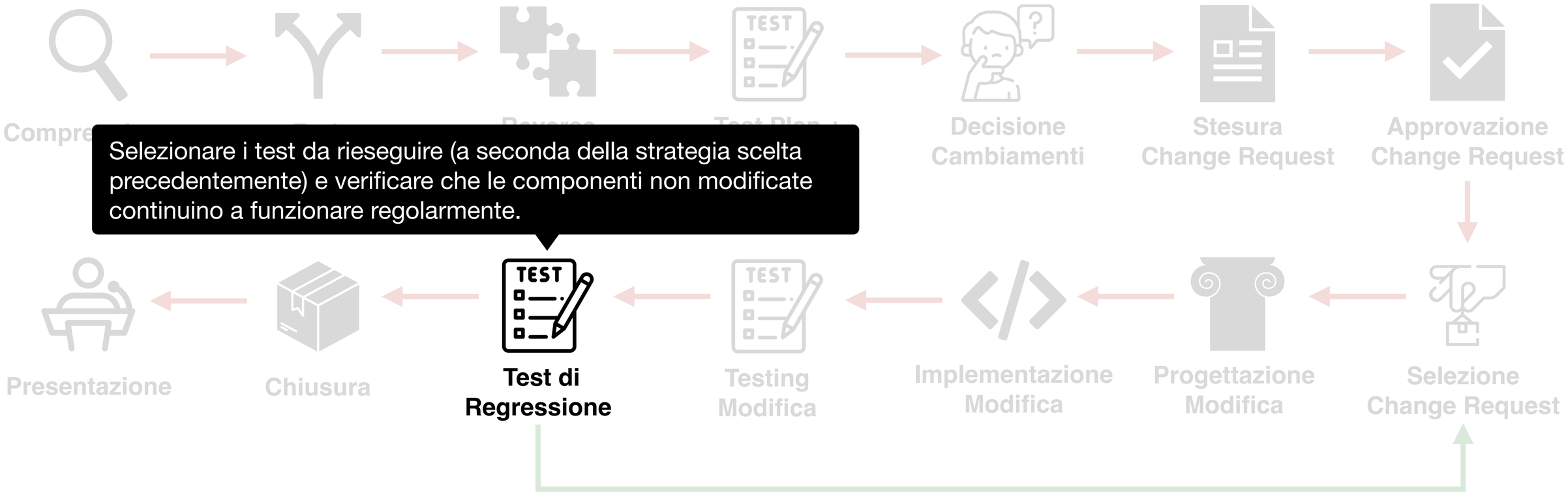
Questo è il processo che si **dovrebbe** seguire per il completamento del progetto:



Regole di Ingaggio

In questa presentazione vedremo alcune possibili idee di progetto, che consistono in manutenzione ed evoluzione di applicativi esistenti. Questi applicativi possono essere tool/script di analisi/manutenzione del codice e modelli predittivi per assistere attività di sviluppo.

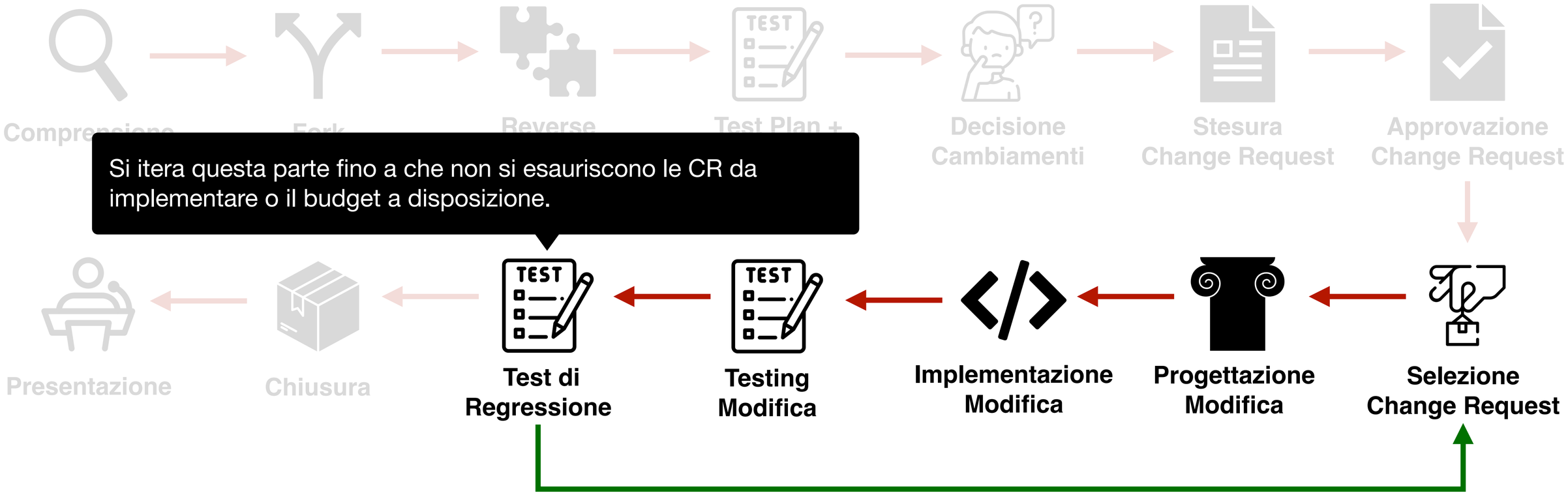
Questo è il processo che si **dovrebbe** seguire per il completamento del progetto:



Regole di Ingaggio

In questa presentazione vedremo alcune possibili idee di progetto, che consistono in manutenzione ed evoluzione di applicativi esistenti. Questi applicativi possono essere tool/script di analisi/manutenzione del codice e modelli predittivi per assistere attività di sviluppo.

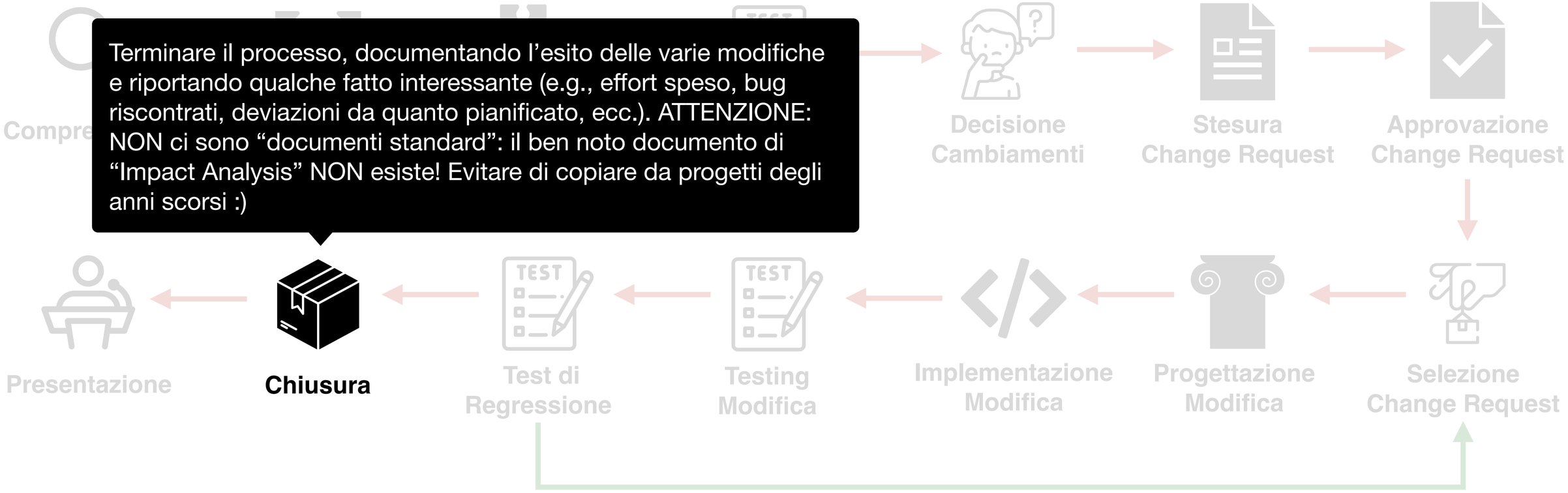
Questo è il processo che si **dovrebbe** seguire per il completamento del progetto:



Regole di Ingaggio

In questa presentazione vedremo alcune possibili idee di progetto, che consistono in manutenzione ed evoluzione di applicativi esistenti. Questi applicativi possono essere tool/script di analisi/manutenzione del codice e modelli predittivi per assistere attività di sviluppo.

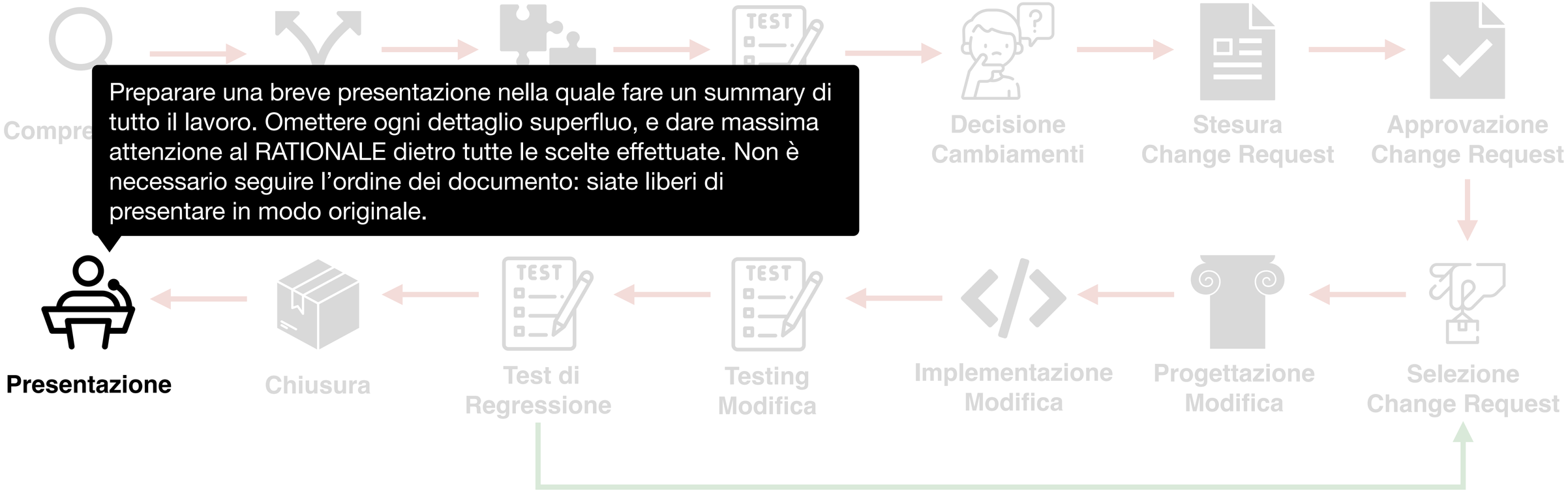
Questo è il processo che si **dovrebbe** seguire per il completamento del progetto:



Regole di Ingaggio

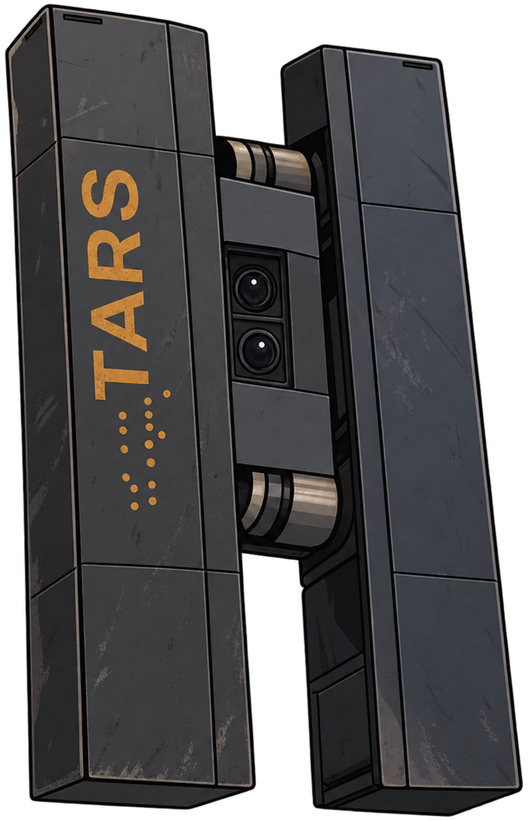
In questa presentazione vedremo alcune possibili idee di progetto, che consistono in manutenzione ed evoluzione di applicativi esistenti. Questi applicativi possono essere tool/script di analisi/manutenzione del codice e modelli predittivi per assistere attività di sviluppo.

Questo è il processo che si **dovrebbe** seguire per il completamento del progetto:



TARS

TARS è un'estensione per VS Code che utilizza un LLM per spiegare il codice direttamente nell'editor, adattando la profondità e il tono al profilo dello sviluppatore (esperienza, ruolo, preferenze) grazie a un approccio di Theory of Mind e ancorandosi alla documentazione del progetto tramite RAG.



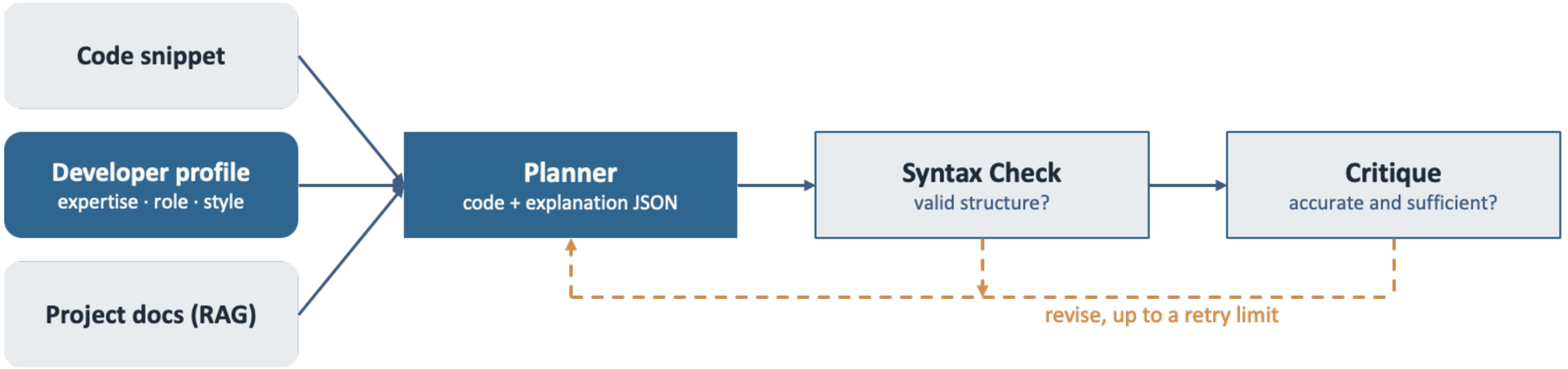
Repository: https://github.com/leotodisco/tars

Paper: https://atdepo.github.io/assets/papers/C13.pdf

Tutor: Antonio Della Porta
Email: adellaporta@unisa.it

TARS





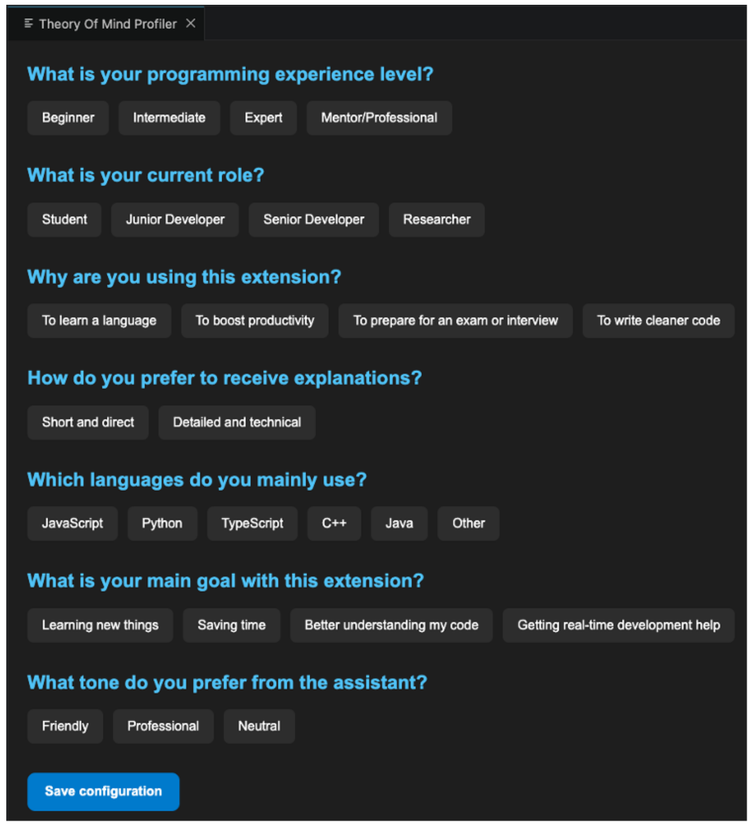
Repository: https://github.com/leotodisco/tars

Paper: https://atdepo.github.io/assets/papers/C13.pdf

Tutor: Antonio Della Porta
Email: adellaporta@unisa.it

TARS





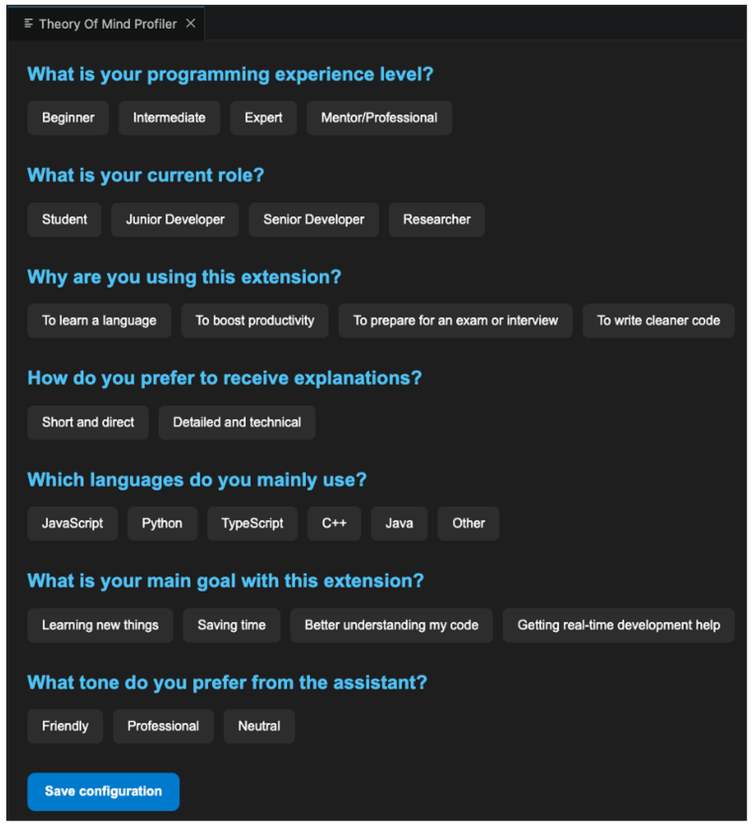
Repository: https://github.com/leotodisco/tars

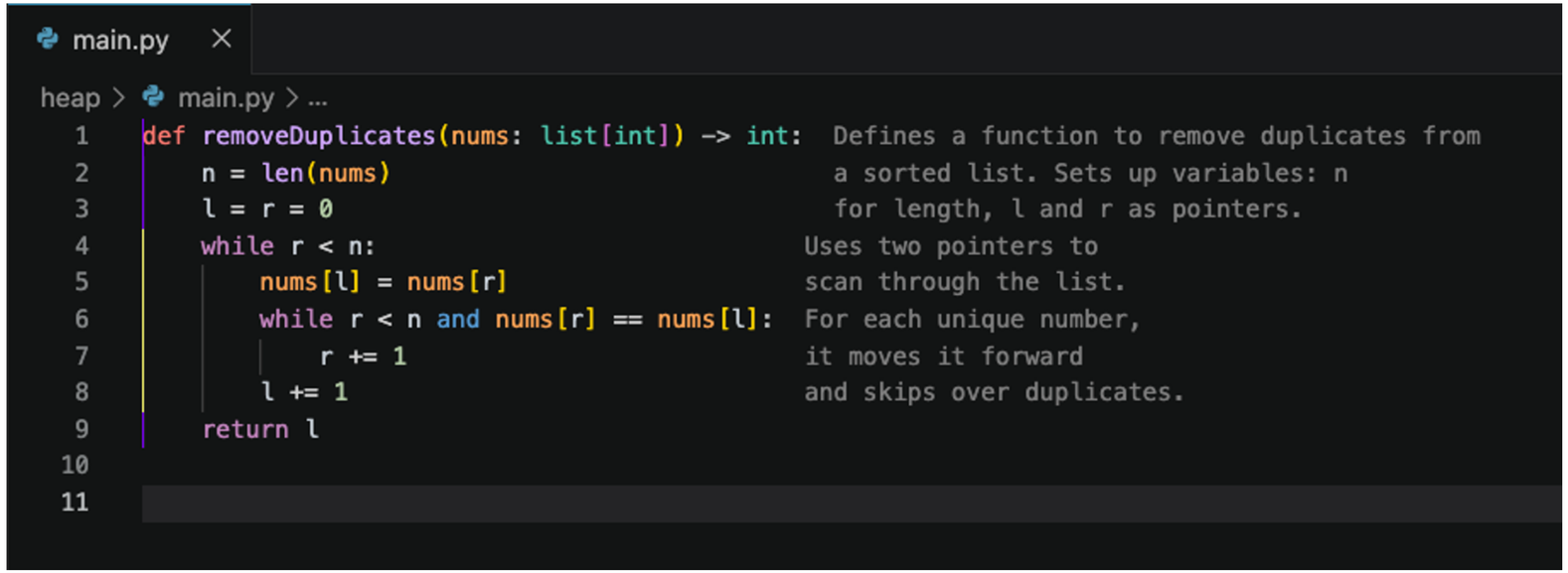
Paper: https://atdepo.github.io/assets/papers/C13.pdf

Tutor: Antonio Della Porta
Email: adellaporta@unisa.it

TARS







Repository: https://github.com/leotodisco/tars

Paper: https://atdepo.github.io/assets/papers/C13.pdf

Tutor: Antonio Della Porta
Email: adellaporta@unisa.it

TARS



Idea #1: Refactoring della GUI

L'interfaccia attuale di TARS è essenziale e poco rifinita. Un redesign con spiegazioni visivamente più chiare e un questionario di profilazione più guidato renderebbe l'esperienza più piacevole e potrebbe favorire l'adozione dello strumento.

Idea #2: Supportare LLM open-weight.

TARS oggi dipende da modelli proprietari accessibili via API (GPT-4.1 Nano nella valutazione), con costi e vincoli di privacy sul codice inviato. Integrare modelli open-weight eseguibili in locale permetterebbe di mantenere il codice sull'infrastruttura dello sviluppatore e di rendere lo strumento utilizzabile anche in contesti aziendali sensibili.

Repository: https://github.com/leotodisco/tars

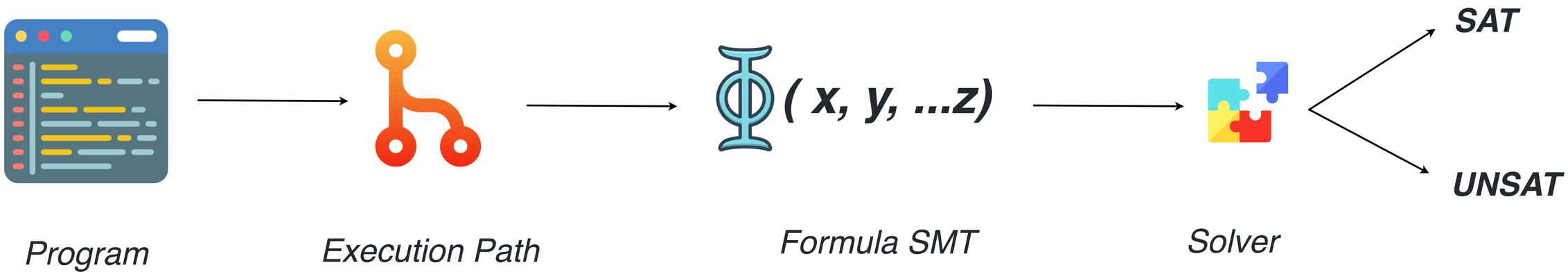
Paper: https://atdepo.github.io/assets/papers/C13.pdf

Tutor: Antonio Della Porta
Email: adellaporta@unisa.it

InterSect

Interval-guided sampler for SMT formula

InterSect è uno strumento progettato per eseguire il campionamento uniforme (o quasi-uniforme) delle soluzioni ammissibili (satisfying assignments o modelli) di formule SMT (Satisfiability Modulo Theories).



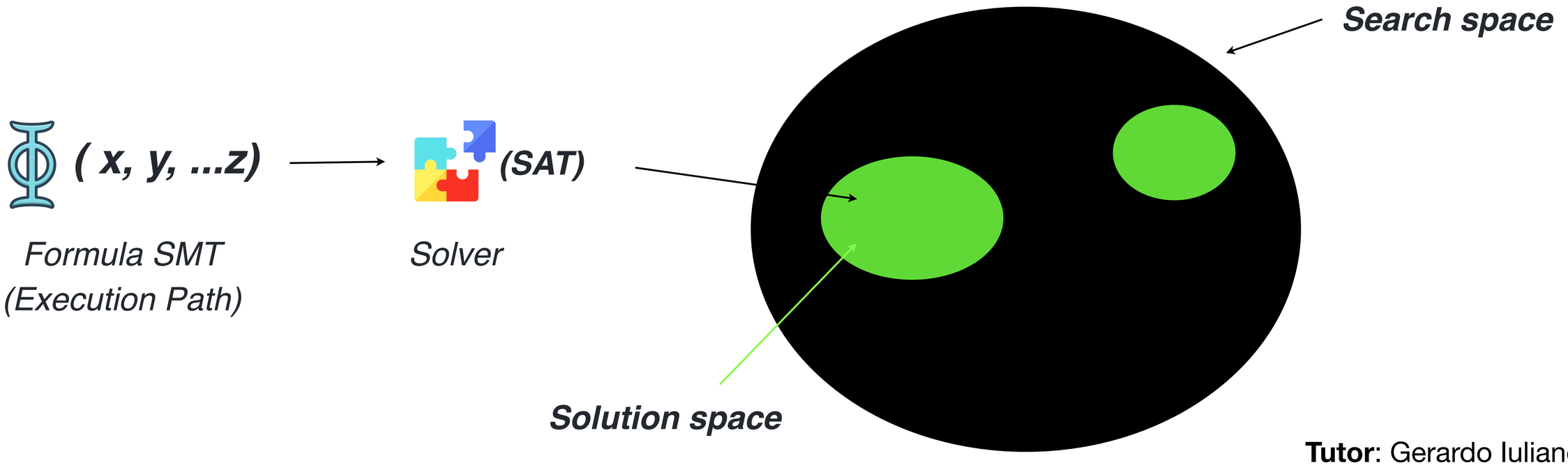
Repository: Contattare il tutor per accedere alla repo privata

Tutor: Gerardo Iuliano
Email: geiuliano@unisa.it

InterSect

Interval-guided sampler for SMT formula

InterSect è uno strumento progettato per eseguire il campionamento uniforme (o quasi-uniforme) delle soluzioni ammissibili (satisfying assignments o modelli) di formule SMT (Satisfiability Modulo Theories).



Repository: Contattare il tutor per accedere alla repo privata

Tutor: Gerardo Iuliano
Email: geiuliano@unisa.it

InterSect

Interval-guided sampler for SMT formula

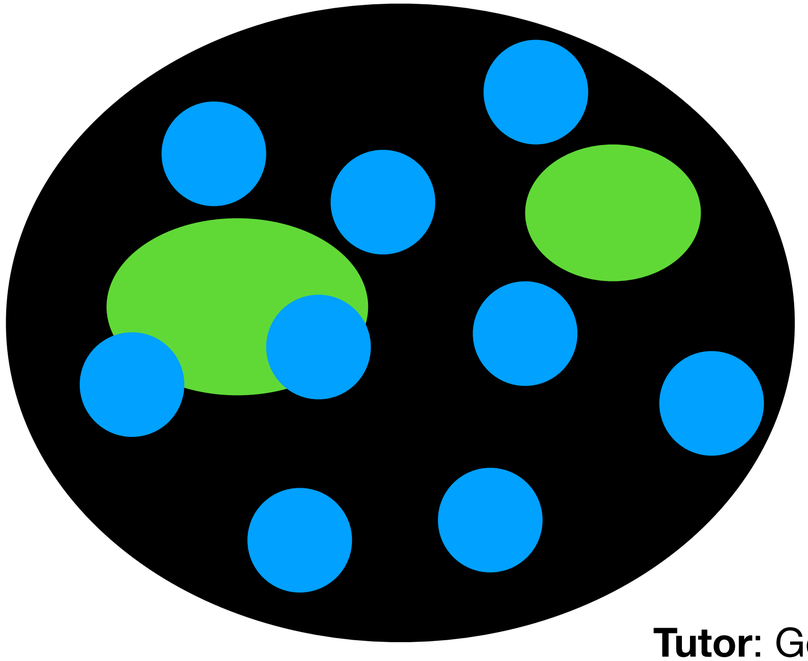
InterSect è uno strumento progettato per eseguire il campionamento uniforme (o quasi-uniforme) delle soluzioni ammissibili (satisfying assignments o modelli) di formule SMT (Satisfiability Modulo Theories).

InterSect

Input: formula SMT

1-Genera intervalli nello spazio
2-Lancia il Solver sugli intervalli
3-Se interseca -> SAT -> Soluzione
4-Altrimenti riparti da 1

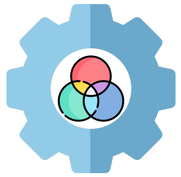
Output: soluzioni (samples)



Repository: Contattare il tutor per accedere alla repo privata

Tutor: Gerardo Iuliano
Email: geiuliano@unisa.it

InterSect



Interval-guided sampler for SMT formula

Idea #1

Refactoring del codice. InterSect è implementato in Python senza nessun design pattern e senza seguire best practices.
Goal: renderlo modulare, facilmente manutenibile, e portabile (Docker)

Idea #2

Aggiugere una GUI. InterSect espone una CLI tradizionale.
Goal: trasformare InterSect in una web-app o simile implementando una GUI che ne semplifica l'interazione. Usare Docker per portabilità.

Repository: Contattare il tutor per accedere alla repo privata

Tutor: Gerardo Iuliano
Email: geiuliano@unisa.it

FADE

Fairness Assessment under Dataset Evolution: stress testing dei dataset ML tramite mutazioni

FADE è un framework Python che individua le feature *fairness-fragile* di un dataset prima dell'addestramento. Applica operatori di mutazione (numerici, categorici, testuali) a ogni feature, ricalcola i *bias symptoms* a livello di dati e classifica le feature con un Composite Fragility Score. Non richiede di addestrare modelli e porta la valutazione di fairness da post-hoc a preventiva, simulando il data drift in pipeline MLOps.

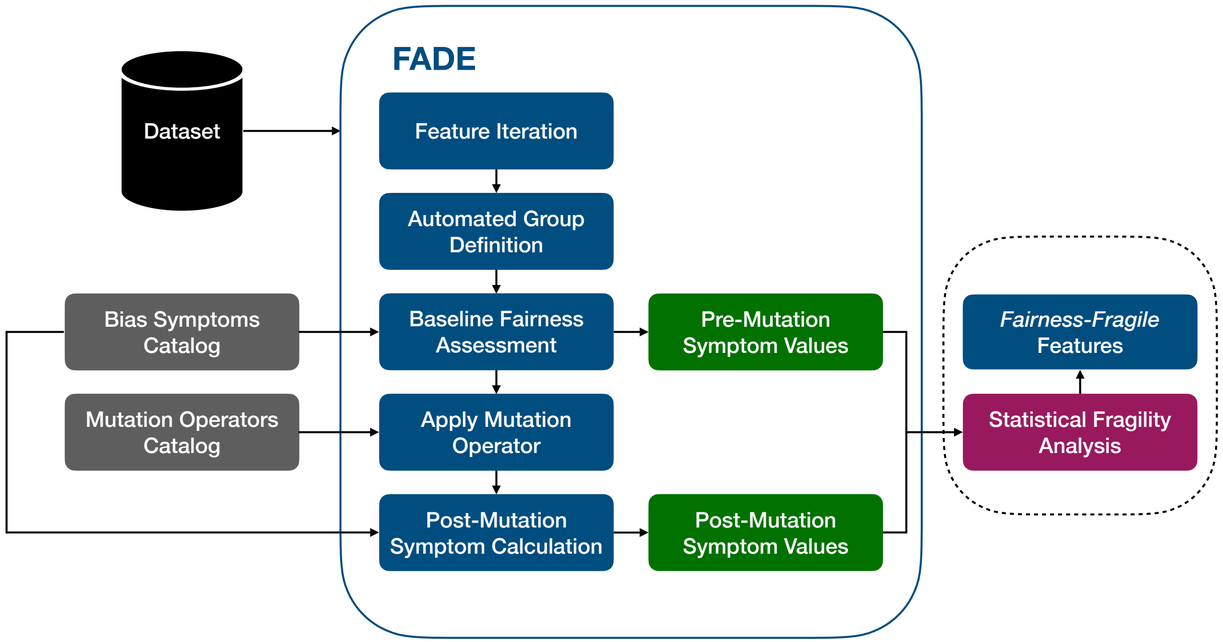
Repository: https://github.com/alfcan/fade

Paper: https://papers.ssrn.com/sol3/papers.cfm?abstract_id=6731297

Tutor: Alfonso Cannavale
Email: acannavale@unisa.it

FADE

Fairness Assessment under Dataset Evolution: stress testing dei dataset ML tramite mutazioni



Repository: https://github.com/alfcan/fade

Paper: https://papers.ssrn.com/sol3/papers.cfm?abstract_id=6731297

Tutor: Alfonso Cannavale
Email: acannavale@unisa.it

FADE

Fairness Assessment under Dataset Evolution: stress testing dei dataset ML tramite mutazioni

Idea #1: Refactoring per estendibilità e CLI

Ristrutturare il codice in modo che operatori di mutazione e bias symptom siano componenti registrabili e configurabili. Introdurre una CLI non interattiva (parametri e file di configurazione, seed riproducibile) e dimostrare l'estendibilità aggiungendo un nuovo operatore e un nuovo symptom.

Idea #2: Modulo di verifica model-level

Aggiungere a FADE una fase di verifica che addestra classificatori sul dataset originale e su quelli mutati e confronta le metriche di fairness. Il modulo deve restituire un report che indica se la fragilità rilevata a livello di dati si traduce in variazioni significative nel comportamento del modello.

Repository: https://github.com/alfcan/fade

Paper: https://papers.ssrn.com/sol3/papers.cfm?abstract_id=6731297

Tutor: Alfonso Cannavale
Email: acannavale@unisa.it

ChronoCode

Analisi dei code smell Python lungo la storia dei commit: AI vs Human

ChronoCode è un tool di analisi statica che misura come evolvono i code smell in un progetto Python. Confronta uno snapshot con la revisione precedente oppure percorre l'intera storia dei commit, usando DPy per rilevare gli smell e StaticTracker per tracciare quali vengono introdotti o rimossi. I commit sono classificati come AI-generated o Human-authored tramite euristiche, e i risultati sono aggregati in riepiloghi e grafici.

Repository: https://github.com/gilbertrec/ChronoCode

Tutor: Gilberto Recupito
Email: gilberto.recupito@vub.be
Tutor: Alfonso Cannavale
Email: acannavale@unisa.it

ChronoCode

Analisi dei code smell Python lungo la storia dei commit: AI vs Human

ChronoCode è un tool di analisi statica che misura come evolvono i code smell in un progetto Python. Confronta uno snapshot con la revisione precedente oppure percorre l'intera storia dei commit, usando DPy per rilevare gli smell e StaticTracker per tracciare quali vengono introdotti o rimossi. I commit sono classificati come AI-generated o Human-authored tramite euristiche, e i risultati sono aggregati in riepiloghi e grafici.

Idea #1: Analizzatori statici pluggabili

Generalizzare l'architettura, oggi legata a DPy, in modo che qualsiasi analizzatore statico (smell, vulnerabilità, warning) si integri tramite un adapter basato su SmellInstance. Integrare almeno un nuovo tool (es. Bandit per le vulnerabilità o Pylint) e confrontare il trend AI vs Human.

Idea #2: Metriche di codice nel tempo

Aggiungere il monitoraggio di metriche (es. complessità ciclomatica, LOC, maintainability index) lungo la storia dei commit, con aggregazioni e grafici per autore AI vs Human, in un livello separato dagli smell.

Repository: https://github.com/gilbertrec/ChronoCode

Tutor: Gilberto Recupito
Email: gilberto.recupito@vub.be
Tutor: Alfonso Cannavale
Email: acannavale@unisa.it

ChronoCode

Analisi dei code smell Python lungo la storia dei commit: AI vs Human

ChronoCode è un tool di analisi statica che misura come evolvono i code smell in un progetto Python. Confronta uno snapshot con la revisione precedente oppure percorre l'intera storia dei commit, usando DPy per rilevare gli smell e StaticTracker per tracciare quali vengono introdotti o rimossi. I commit sono classificati come AI-generated o Human-authored tramite euristiche, e i risultati sono aggregati in riepiloghi e grafici.

Idea #3: Metriche di produttività e di processo

Estendere con metriche di processo calcolate dalla storia Git: code churn (righe aggiunte e rimosse), commit per settimana, ownership dei file (numero di autori e quota del contributore principale). Le metriche vanno visualizzate nel tempo e confrontate tra commit AI e Human.

Repository: https://github.com/gilbertrec/ChronoCode

Tutor: Gilberto Recupito
Email: gilberto.recupito@vub.be
Tutor: Alfonso Cannavale
Email: acannavale@unisa.it

RefactoringMiner

Rilevamento di refactoring e AST diff multi-linguaggio

Libreria Java (con CLI, Docker, plugin Chrome e server MCP) che rileva i refactoring applicati nella storia di un progetto e genera AST diff refactoring-aware a livello di commit, pull request e range di commit. Supporta Java, Python, Kotlin, TypeScript/JavaScript e C/C++ e riconosce oltre 100 tipi di refactoring.

Idea: Supporto a C# (MVP)

Estendere RefactoringMiner a C# integrando un parser nell'infrastruttura multi-linguaggio già usata per gli altri linguaggi. L'obiettivo è rilevare un sottoinsieme di refactoring (es. Rename/ Move Method e Class, Change Return Type) su commit C# reali, con test automatici.

Repository: https://github.com/tsantalis/RefactoringMiner

Paper: https://ieeexplore.ieee.org/abstract/document/9136878

Tutor: Gilberto Recupito
Email: gilberto.recupito@vub.be
Tutor: Alfonso Cannavale
Email: acannavale@unisa.it